05-3-terraform-fixed

accounts-api security assessment · generated from the repository, not hand-edited

```
// 05-3-terraform-fixed.tf
//
// Section 5 deliverable: the remediated Terraform. Defects and their
// exploitability ranking are in 05-6-findings.md.
//
// Assembled by scripts/build-submission.py from the repository.

// ============================================================================
// SOURCE: terraform/rds.tf
// ============================================================================
// Section 4b, findings B1–B4. Threat T3.
// Diff against policy/fixtures/terraform/bad/main.tf.

resource "aws_db_instance" "accounts" {
  identifier     = "accounts-prod"
  engine         = "postgres"
  engine_version = "16.4"
  instance_class = "db.r6g.large"

  // B1: no public endpoint, private subnets only. Combined with the security
  // group in security_groups.tf, the only route to 5432 is from the EKS nodes.
  publicly_accessible    = false
  db_subnet_group_name   = var.db_subnet_group_name
  vpc_security_group_ids = [aws_security_group.db.id]

  // B2: no `password` argument anywhere. RDS creates and owns the master
  // credential in Secrets Manager, so it never appears in Terraform state.
  // The application does not use this credential at all — it authenticates with
  // IAM tokens (below) and reads a separate, application-scoped secret.
  username                            = "rdsadmin"
  manage_master_user_password         = true
  master_user_secret_kms_key_id       = aws_kms_key.secrets.arn
  iam_database_authentication_enabled = true

  // B3: encrypted at rest with a customer-managed key. Cannot be enabled in
  // place later, which is why it is not something to defer.
  storage_encrypted = true
  kms_key_id        = aws_kms_key.rds.arn
  storage_type      = "gp3"
  allocated_storage = 200

  // B4: recoverable. 35 days is the RDS maximum for automated backups and lines
  // up with the incident-discovery window we assume in DECISIONS.md §3.
  backup_retention_period   = 35
  backup_window             = "18:30-19:30" // 00:00 IST
  copy_tags_to_snapshot     = true
  delete_automated_backups  = false
  skip_final_snapshot       = false
  final_snapshot_identifier = "accounts-prod-final"
  deletion_protection       = true

  // Evidence and hygiene
  performance_insights_enabled          = true
  performance_insights_kms_key_id       = aws_kms_key.rds.arn
  performance_insights_retention_period = 731
  monitoring_interval                   = 30
  monitoring_role_arn                   = aws_iam_role.rds_monitoring.arn
```

```
  enabled_cloudwatch_logs_exports       = ["postgresql", "upgrade"]

  auto_minor_version_upgrade = true
  maintenance_window         = "sun:19:30-sun:20:30"
  ca_cert_identifier         = "rds-ca-rsa2048-g1"
  multi_az                   = true
  parameter_group_name       = aws_db_parameter_group.accounts.name
  apply_immediately          = false
}

// Forces TLS on every client connection. Without rds.force_ssl the application
// can be talked out of TLS by a network-position attacker, and the encryption
// in transit claim in docs/06 would be aspirational.
resource "aws_db_parameter_group" "accounts" {
  name   = "accounts-prod-pg16"
  family = "postgres16"

  parameter {
    name         = "rds.force_ssl"
    value        = "1"
    apply_method = "pending-reboot"
  }

  parameter {
    name  = "log_statement"
    value = "ddl" // never `all`: statement logs would carry IBANs into CloudWatch
  }
}

resource "aws_iam_role" "rds_monitoring" {
  name = "accounts-prod-rds-monitoring"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect    = "Allow"
      Principal = { Service = "monitoring.rds.amazonaws.com" }
      Action    = "sts:AssumeRole"
    }]
  })
}

resource "aws_iam_role_policy_attachment" "rds_monitoring" {
  role       = aws_iam_role.rds_monitoring.name
  policy_arn = "arn:aws:iam::aws:policy/service-role/AmazonRDSEnhancedMonitoringRole"
}

// ============================================================================
// SOURCE: terraform/security_groups.tf
// ============================================================================
// Section 4b, finding B1. Threat T3.

resource "aws_security_group" "db" {
  name        = "accounts-prod-db"
  description = "accounts-api PostgreSQL. Ingress from the EKS node group only."
  vpc_id      = var.vpc_id
}

// The whole fix in one resource: source_security_group_id, not cidr_blocks.
// A CIDR is a statement about the network; a security group id is a statement
// about identity, and it stays correct when the subnets are renumbered.
resource "aws_security_group_rule" "db_ingress" {
  type                     = "ingress"
  description              = "PostgreSQL from EKS nodes running ns/accounts"
```

```
  from_port                = 5432
  to_port                  = 5432
  protocol                 = "tcp"
  source_security_group_id = var.eks_node_security_group_id
  security_group_id        = aws_security_group.db.id
}

// Explicit and narrow. An unqualified egress rule on a database SG is a
// data-exfiltration path that nobody looks at because it is on the "outbound" side.
resource "aws_security_group_rule" "db_egress_none" {
  type              = "egress"
  description       = "No outbound. RDS does not initiate connections."
  from_port         = 0
  to_port           = 0
  protocol          = "-1"
  cidr_blocks       = ["127.0.0.1/32"]
  security_group_id = aws_security_group.db.id
}

// ============================================================================
// SOURCE: terraform/cloudtrail.tf
// ============================================================================
// Section 4b, finding B5. This is the control that every "what evidence would
// you have?" answer in DECISIONS.md §3 depends on.

resource "aws_cloudtrail" "org" {
  name           = "org-trail"
  s3_bucket_name = aws_s3_bucket.trail.id
  s3_key_prefix  = "org"

  // The three flips from the original, in order of what they buy:
  //
  //  include_global_service_events: IAM, STS, CloudFront and Route 53 events are
  //    GLOBAL. With this false they are not recorded anywhere — i.e. the exact API
  //    calls of an identity attack (CreateAccessKey, AttachRolePolicy,
  //    AssumeRole) were invisible. This is the one that mattered most.
  //  is_multi_region_trail: an attacker working in us-east-1 was invisible.
  //  enable_log_file_validation: produces the hourly SHA-256 digest chain. Without
  //    it the log is not tamper-evident and is weak evidence to a regulator.
  include_global_service_events = true
  is_multi_region_trail         = true
  enable_log_file_validation    = true

  is_organization_trail = true
  kms_key_id            = aws_kms_key.trail.arn

  cloud_watch_logs_group_arn = "${aws_cloudwatch_log_group.trail.arn}:*"
  cloud_watch_logs_role_arn  = aws_iam_role.trail_to_cwl.arn

  // Data events for the one bucket that could carry exported customer data, and
  // for Lambda. Deliberately not "all S3 buckets" — that is a large bill for
  // telemetry nobody reads, and DECISIONS.md §4 names the gap it leaves.
  advanced_event_selector {
    name = "customer-data-exports"

    field_selector {
      field  = "eventCategory"
      equals = ["Data"]
    }

    field_selector {
      field  = "resources.type"
      equals = ["AWS::S3::Object"]
    }
```

```
    field_selector {
      field       = "resources.ARN"
      starts_with = ["arn:aws:s3:::example-bank-accounts-exports/"]
    }
  }
}

// Archive lives in a SEPARATE account. The point is that a principal compromised
// in prod — including a compromised administrator — cannot reach it.
resource "aws_s3_bucket" "trail" {
  bucket = "example-bank-cloudtrail-archive-ap-south-1"

  // Compliance-mode Object Lock: not even the account root can shorten this.
  object_lock_enabled = true
}

resource "aws_s3_bucket_object_lock_configuration" "trail" {
  bucket = aws_s3_bucket.trail.id

  rule {
    default_retention {
      mode = "COMPLIANCE"
      days = 2557 // 7 years
    }
  }
}

resource "aws_s3_bucket_server_side_encryption_configuration" "trail" {
  bucket = aws_s3_bucket.trail.id

  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm     = "aws:kms"
      kms_master_key_id = aws_kms_key.trail.arn
    }
    bucket_key_enabled = true
  }
}

resource "aws_s3_bucket_versioning" "trail" {
  bucket = aws_s3_bucket.trail.id

  versioning_configuration {
    status = "Enabled"
  }
}

resource "aws_s3_bucket_public_access_block" "trail" {
  bucket                  = aws_s3_bucket.trail.id
  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls      = true
  restrict_public_buckets = true
}

resource "aws_cloudwatch_log_group" "trail" {
  name              = "/aws/cloudtrail/org-trail"
  retention_in_days = 400
  kms_key_id        = aws_kms_key.trail.arn
}

resource "aws_iam_role" "trail_to_cwl" {
  name = "cloudtrail-to-cwl"
```

```
  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect    = "Allow"
      Principal = { Service = "cloudtrail.amazonaws.com" }
      Action    = "sts:AssumeRole"
    }]
  })
}

// GuardDuty is the detective control that turns the trail from an archive into
// an alert. Enabled at the org level with the EKS and RDS protection plans,
// because the two threats it actually helps with here are T2 (credential
// exfiltration: the InstanceCredentialExfiltration finding) and anomalous RDS login.
resource "aws_guardduty_detector" "prod" {
  enable                       = true
  finding_publishing_frequency = "FIFTEEN_MINUTES"

  datasources {
    kubernetes {
      audit_logs {
        enable = true
      }
    }
  }
}

// ============================================================================
// SOURCE: terraform/kms.tf
// ============================================================================
// Key custody. See docs/06-secrets-rotation.md for the one-line statement of what
// this actually defends against — and what it does not.
//
// Traceability note: these keys do NOT map to a named threat in THREAT-MODEL.md.
// They are here because key custody is a regulatory obligation (RBI / PCI DSS 3.5
// / DPDP Act) rather than because they mitigate a threat in my top five. Stated as
// such rather than retrofitted to a threat — see TRACEABILITY.md.

resource "aws_kms_key" "rds" {
  description             = "accounts-api RDS storage + performance insights"
  enable_key_rotation     = true // annual automatic rotation of the backing key
  deletion_window_in_days = 30
  multi_region            = false // residency: the key material does not leave ap-south-1
}

resource "aws_kms_alias" "rds" {
  name          = "alias/accounts-api-rds"
  target_key_id = aws_kms_key.rds.key_id
}

resource "aws_kms_key" "secrets" {
  description             = "accounts-api Secrets Manager envelope key"
  enable_key_rotation     = true
  deletion_window_in_days = 30
  multi_region            = false

  // The grant that matters: only the pod role may decrypt, only via Secrets
  // Manager, only for this secret's encryption context. A stolen kms:Decrypt
  // permission is useless against ciphertext from any other source.
  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
```

```
        Sid       = "AccountRootForBreakGlassAdministration"
        Effect    = "Allow"
        Principal = { AWS = "arn:aws:iam::123456789012:root" }
        Action    = ["kms:Describe*", "kms:Get*", "kms:List*", "kms:PutKeyPolicy"]
        Resource  = "*"
      },
      {
        Sid       = "PodRoleDecryptViaSecretsManagerOnly"
        Effect    = "Allow"
        Principal = { AWS = "arn:aws:iam::123456789012:role/accounts-api-pod" }
        Action    = "kms:Decrypt"
        Resource  = "*"
        Condition = {
          StringEquals = {
            "kms:ViaService" = "secretsmanager.ap-south-1.amazonaws.com"
          }
        }
      },
      {
        Sid       = "RotationLambdaEncryptDecrypt"
        Effect    = "Allow"
        Principal = { AWS = "arn:aws:iam::123456789012:role/accounts-api-secret-rotation" }
        Action    = ["kms:Decrypt", "kms:GenerateDataKey"]
        Resource  = "*"
      }
    ]
  })
}

resource "aws_kms_alias" "secrets" {
  name          = "alias/accounts-api-secrets"
  target_key_id = aws_kms_key.secrets.key_id
}

resource "aws_kms_key" "trail" {
  description             = "CloudTrail archive + log group"
  enable_key_rotation     = true
  deletion_window_in_days = 30
  multi_region            = false
}

resource "aws_kms_alias" "trail" {
  name          = "alias/cloudtrail-archive"
  target_key_id = aws_kms_key.trail.key_id
}

// ============================================================================
// SOURCE: terraform/ecr.tf
// ============================================================================
// Threat T4 — artefact integrity. Backs the claims in docs/02-build-integrity.md.

resource "aws_ecr_repository" "accounts_api" {
  name = "accounts-api"

  // The load-bearing line for "immutable tags". ECR rejects a second push to an
  // existing tag at the registry, which is the only layer an attacker with
  // registry credentials cannot argue with. The pipeline check and the Kyverno
  // digest policy are defence in depth around this one setting.
  image_tag_mutability = "IMMUTABLE"

  image_scanning_configuration {
    scan_on_push = true
  }
```

```
  encryption_configuration {
    encryption_type = "KMS"
    kms_key         = aws_kms_key.ecr.arn
  }
}

resource "aws_kms_key" "ecr" {
  description             = "accounts-api ECR layer encryption"
  enable_key_rotation     = true
  deletion_window_in_days = 30
  multi_region            = false
}

// Push is limited to the one GitHub Actions role. Pull is limited to the prod
// account. Separating these means compromising the build account does not give
// you the ability to serve images to anyone else's cluster.
resource "aws_ecr_repository_policy" "accounts_api" {
  repository = aws_ecr_repository.accounts_api.name

  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Sid       = "PushFromGitHubActionsOnly"
        Effect    = "Allow"
        Principal = { AWS = "arn:aws:iam::210987654321:role/gha-accounts-api-ecr-push" }
        Action = [
          "ecr:PutImage",
          "ecr:InitiateLayerUpload",
          "ecr:UploadLayerPart",
          "ecr:CompleteLayerUpload",
          "ecr:BatchCheckLayerAvailability",
          "ecr:DescribeImages"
        ]
      },
      {
        Sid       = "PullFromProdClusterOnly"
        Effect    = "Allow"
        Principal = { AWS = "arn:aws:iam::123456789012:root" }
        Action = [
          "ecr:GetDownloadUrlForLayer",
          "ecr:BatchGetImage",
          "ecr:BatchCheckLayerAvailability"
        ]
      },
      {
        // Even the push role may not remove an image. Deleting a digest is how
        // you break the audit trail between "what we scanned" and "what ran".
        Sid       = "NobodyDeletesImages"
        Effect    = "Deny"
        Principal = "*"
        Action    = ["ecr:BatchDeleteImage", "ecr:DeleteRepository", "ecr:DeleteRepositoryPolicy"]
      }
    ]
  })
}

// Retention, not deletion-on-demand: expiry is a scheduled, auditable policy
// rather than an API call an attacker can make.
resource "aws_ecr_lifecycle_policy" "accounts_api" {
  repository = aws_ecr_repository.accounts_api.name

  policy = jsonencode({
    rules = [{
```

```
      rulePriority = 1
      description  = "Keep the last 200 images; prod rollback window plus audit."
      selection = {
        tagStatus   = "any"
        countType   = "imageCountMoreThan"
        countNumber = 200
      }
      action = { type = "expire" }
    }]
  })
}

// ================================================================================
// SOURCE: terraform/variables.tf
// ================================================================================
variable "vpc_id" {
  type        = string
  description = "VPC hosting the EKS cluster and the database subnet group."
  default     = "vpc-0aa11bb22cc33dd44"
}

variable "eks_node_security_group_id" {
  type        = string
  description = "Security group attached to the EKS node group. The ONLY source permitted to reach
5432."
  default     = "sg-0node11223344556"
}

variable "db_subnet_group_name" {
  type        = string
  description = "Private, no-route-to-IGW subnets."
  default     = "accounts-prod-private"
}

variable "log_archive_account_id" {
  type        = string
  description = "Separate AWS account holding the immutable CloudTrail archive. Deliberately not
the prod account: the point of the archive is that a compromised prod principal cannot reach it."
  default     = "444455556666"
}

# NOTE: there is deliberately no `db_password` variable. See rds.tf — the master
# credential is created and rotated by RDS in Secrets Manager, so it never enters
# Terraform state, CI variables, or anyone's shell history. This absence is the fix
# for finding B2 in docs/04-findings.md.

// ================================================================================
// SOURCE: terraform/versions.tf
// ================================================================================
terraform {
  required_version = ">= 1.9.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.80"
    }
  }

  # State holds the plan, not the credentials — see rds.tf: the DB password is
  # managed by RDS in Secrets Manager, so it is never in this file.
  backend "s3" {
    bucket       = "example-bank-tfstate-ap-south-1"
    key          = "accounts-api/prod/terraform.tfstate"
```

```
    region       = "ap-south-1"
    encrypt      = true
    kms_key_id   = "arn:aws:kms:ap-south-1:123456789012:key/tfstate"
    use_lockfile = true
  }
}

provider "aws" {
  region = "ap-south-1" # RBI localisation: this stack does not exist elsewhere

  default_tags {
    tags = {
      Service            = "accounts-api"
      Environment        = "production"
      DataClassification = "pii-and-cardholder-adjacent"
      Owner              = "retail-banking-platform"
      ThreatModel        = "THREAT-MODEL.md"
    }
  }
}
```